

PROYECTO 2: DISEÑO Y ANÁLISIS DE ALGORITMOS

AUTORES

- William Pollock, 202221321
- Nicolás Hernández, 202322148

ALGORITMO DE SOLUCIÓN

Para resolver este problema decidimos usar BFS como nuestro algoritmo de recorrido, ya que su manera de recorrer el grafo por capas nos parece más conveniente que usar rutas más largas primero y recorrer la mayoría del grafo antes de descubrir la óptima (lo cual ocurriría si fuéramos a usar DFS).

Modelamos la posición de Samus en las plataformas con la tupla (*posición*, *energía*), donde:

- *posición* representa la plataforma actual donde está parada Samus ($0 \leq \text{posición} \leq n$), n siendo el número total de plataformas
- *energía* representa la energía total que ya ha sido consumida en el momento actual ($0 \leq \text{energía} \leq E$), E siendo el número inicial de energía disponible.

Antes de llegar a esta solución, teníamos pensado implementar otra manera de resolver el problema. En un principio, intentamos solucionar el problema implementando un algoritmo de programación dinámica. Nuestra idea era guardar en un arreglo de n elementos las soluciones del problema para todos los posibles números de plataformas desde 2 hasta n , e ir actualizando estos valores utilizando los valores anteriores del arreglo. A pesar de que creíamos que era una buena solución, nos dimos cuenta de que esta manera de resolver el problema no sirve por una razón fundamental. Esta razón es que la solución para un número determinado n de plataformas no depende en lo absoluto de la solución para $n-1$ plataformas, ni de ninguna solución con menos plataformas. Esto es debido a que en la descripción del problema se indica que Samus puede saltar hacia adelante o hacia atrás, y también puede usar los poderes hacia adelante o hacia atrás. Además, al añadir cada plataforma, deberíamos tener en cuenta si esta tiene algún poder o un robot asesino, por lo que se darían varios casos distintos al añadir cada plataforma. Es por esto que concluimos que la mejor opción no es un algoritmo de programación dinámica, sino un algoritmo basado en grafos.

BÚSQUEDA POR CAPAS

La razón principal por la cual decidimos escoger BFS como nuestro algoritmo de recorrido fue porque explora el grafo capa por capa; esto significa que todas las plataformas a las que se puedan llegar en 1 acción son parte de la primera capa, 2 acciones la segunda, etc. Creamos el arreglo *mejorEnergia[]* para así poder guardar el menor consumo de energía de cada posición.

Samus se puede mover de capa en capa de las siguientes maneras:

- Caminando (C+/C-): $\text{posición} \rightarrow \text{posición} \pm 1$. No tiene costo de energía.
- Saltando/usando poder (S+/S-): $\text{posición} \rightarrow \text{posición} \pm k$, siempre y cuando *posición* tenga el poder k .

- Teletransportándose ($T \pm distancia$): $posición \rightarrow q$, q representando cualquier valor cuya distancia $|q - posición|$ sea menor o igual a la energía restante.

La salida del código se encargará de mostrarle al usuario el mínimo número de acciones (capas) que fueron usadas y además le mostrará la decisión tomada en cada capa. El primer valor es un entero y representa el número de acciones, mientras que los demás valores representan cómo se movió Samus en cada acción/capa.

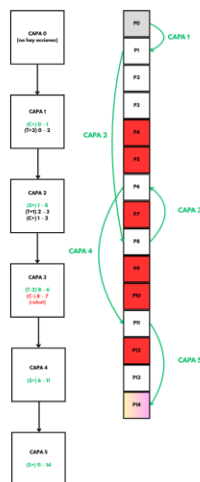
IMPLEMENTACIÓN

Para implementar el problema se usó un *TreeSet libres* para guardar todas las plataformas que no tuvieran robots y que no han sido visitadas todavía. Además, se implementaron 4 arreglos diferentes: *mejorEnergia* para guardar la menor energía gastada al llegar a cada plataforma; *parent* para guardar los caminos tomados en la ruta de Samus; *acc* para guardar las acciones empleadas (C, S o T); y *numAcciones* para indicar la capa a la que llegó BFS al final del procedimiento. Además, también se usó una cola de tipo *ArrayDeque* para ir guardando las plataformas de la capa actual y las de las capas siguientes en orden FIFO (las primeras que entran salen primero). El ciclo dentro de *solucion* usa todo lo anterior para extraer la plataforma actual donde está Samus, detectar cuando llegue a Raven Beak (la última plataforma) y consultar la energía mínima que se gastó para llegar a esa plataforma.

En cuanto a la salida del código, se usa *reconstruirRuta* y Scanner para buscar el predecesor de cada plataforma y las acciones que fueron usadas para llegar hasta a Raven Beak. Se retrocede desde la última plataforma hasta la plataforma inicial (con ayuda de *parent*) y se van poniendo las acciones en una pila. Finalmente, se invierte la pila (para que estén en el orden correcto), se ingresa el número de acciones y se devuelve la línea de salida.

GRÁFICA

Para demostrar el funcionamiento del código, aquí hay una gráfica basada en un ejemplo donde hay 14 plataformas, 2 valores de energía, robots en las plataformas rojas y saltos en la plataforma 1(± 7), 3(± 2), 6(± 5) y 11(± 3).



Lo que ocurre en esta gráfica es que primero se analizan las opciones disponibles por cada capa y se toma alguna de las opciones válidas. El código se asegura de tomar primero las opciones que las lleven

lo más cerca posible a Raven Beak. Por ejemplo, en la capa 2 el código prefiere saltar de 1 hasta 8 en vez de caminar de 1 hasta 2 ya que la primera está mucho más cerca de la plataforma 14.

COMPLEJIDAD TEMPORAL DEL ALGORITMO

Para hallar la complejidad temporal del algoritmo, debemos analizar detalladamente el código. En primer lugar, iniciamos un ciclo para leer cada caso del archivo de entradas, y dentro de este ciclo invocamos al método `solucion()`, para obtener la solución del problema en cada caso. Luego, en la función `solucion()`, implementamos una búsqueda de tipo BFS con una cola de prioridad como estructura de datos. Note que dentro de la búsqueda BFS, hacemos tres ciclos. Los primeros dos solo se recorren dos veces, para evaluar los movimientos adyacentes y los saltos con poder. Como solo recorreremos dos veces estos ciclos, aportan complejidad constante $O(1)$ a la complejidad total. Por otro lado, en el tercer ciclo iteramos una vez por cada elemento en el `TreeSet` libres, por lo que este ciclo aporta $O(n)$ a la complejidad total. Por lo tanto, la complejidad de la función `solucion()` aporta $O(n \log n)$, donde n es el número de plataformas, ya que esta es la complejidad de BFS, y esta complejidad domina a las complejidades de los ciclos interiores. Por último, observe que la función `reconstruirRuta()` aporta complejidad lineal ya que tiene un ciclo que itera una vez por cada elemento del arreglo `parent`. Por lo tanto, note que, de las complejidades mencionadas, la dominante es la de la función `solucion()`, es decir, $O(n \log n)$. Esto quiere decir que la complejidad temporal del algoritmo para cada caso de prueba es $O(n \log n)$, donde n es el número de plataformas.

COMPLEJIDAD ESPACIAL DEL ALGORITMO

Para hallar la complejidad espacial del algoritmo, debemos analizar detalladamente las estructuras de datos inicializadas en el código. Note que las estructuras de datos creadas para la solución del problema fueron: `boolean[] robot`, `int[] poder`, `int[] mejorEnergia`, `int[] parent`, `String[] acc`, `int[] numAcciones`, `TreeSet<Integer> libres` y una cola `ArrayDeque<Integer> adq`. Observe que todas estas estructuras aportan complejidad espacial $O(n)$, ya que todos los arreglos fueron creados con $n+1$ casillas, y el `TreeSet` y la cola de prioridad tienen a lo sumo n elementos. Por lo tanto, como estas fueron las únicas estructuras de datos creadas, concluimos que la complejidad espacial del algoritmo es $O(n)$, donde n es el número de plataformas.